

实验一 新建 MDK 工程项目

1.1 实验目的

新建 MDK 工程项目是学会使用 MDK 的第一步，本实验将讲解新建工程项目步骤，希望同学们了解并熟悉。之后的项目创立都可以根据建立好的项目文件进行修改，以此创建新工程项目。

1.2 实验步骤

（1）首先在桌面上建立一个工程根目录文件夹 `Template`，后续可根据项目名称更改文件夹名，后续的工程文件都将在这个文件夹里建立。然后在工程根目录文件夹下建立以下几个文件夹，新建完成以后，最后得到工程根目录文件夹如图所示。`CORE` 用来存放核心文件和启动文件，`OBJ` 是用来存放编译过程文件以及 `hex` 文件。`FWLib` 文件夹顾名思义用来存放 `ST` 官方提供的库函数源码文件。已有的 `USER` 目录除了用来放工程文件外，还用来存放主函数文件 `main.c`，以及其他包括 `system_stm32f40x.c` 等等。`SYSTEM` 文件夹存放的是我们常用的共用代码，包含延时函数，`IO` 口位操作以及串口相关函数等。

名称	修改日期	类型	大小
CORE	2026/6/28 下午 8:55	文件夹	
FWLIB	2026/6/28 下午 8:56	文件夹	
HARDWARE	2026/6/28 下午 7:27	文件夹	
OBJ	2026/6/28 下午 8:53	文件夹	
SYSTEM	2026/6/28 下午 7:27	文件夹	
USER	2026/6/28 下午 10:48	文件夹	

图 1.1 文件夹名称

（2）打开 MDK，点击 MDK 的菜单：`Project ->New Uvision Project`，然后将目录定位到刚才建立的文件夹 `Template` 之下，在这个目录下面点击子文件夹 `USER`，然后定位到 `USER` 目录下面，我们的工程文件就都保存到 `USER` 文件夹下面。工程命名为 `Template`，点击保存。

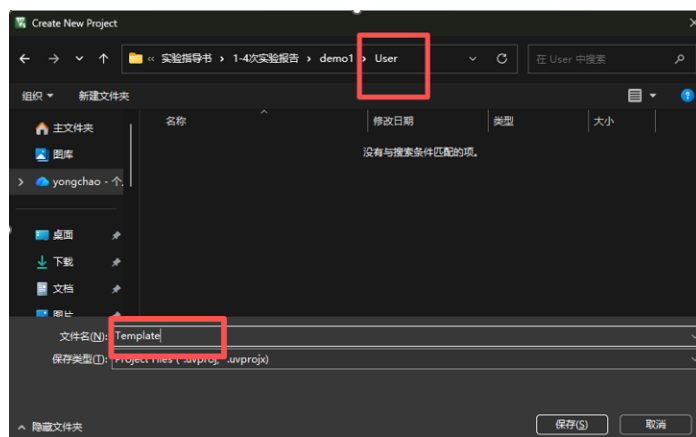
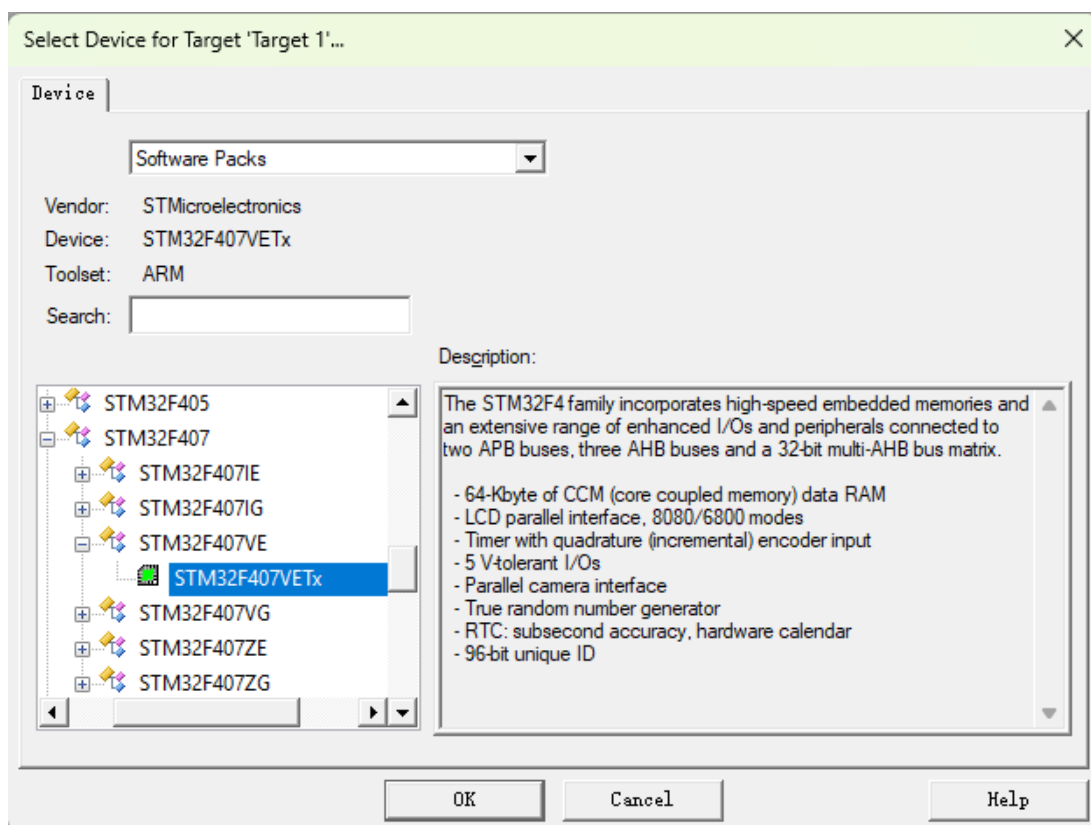


图 1.2 定义工程名称

(3) 接下来会出现一个选择 CPU 的界面，就是选择我们的芯片型号。如图 1.3 所示，由于小车使用的单片机型号为 STM32F407vet6，因此，选择单片机型号为 STM32F407vet6（如果使用的是其他系列的芯片，选择相应的型号就可以了，特别注意：一定要安装对应的 器件 pack 才会显示这些内容!!）



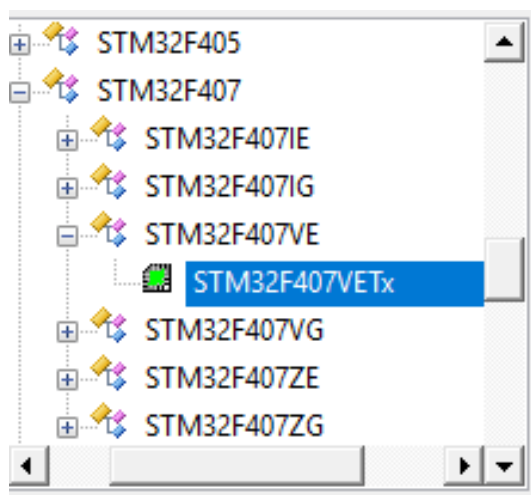


图 1.3 单片机型号选择

点击 OK，MDK 会弹出 Manage Run-Time Environment 对话框，如图 1.4 所示，这是 MDK5 新增的一个功能，在这个界面，我们可以添加自己需要的组件，从而方便构建开发环境，不过这里不做介绍，在所界面，我们直接点击 Cancel。

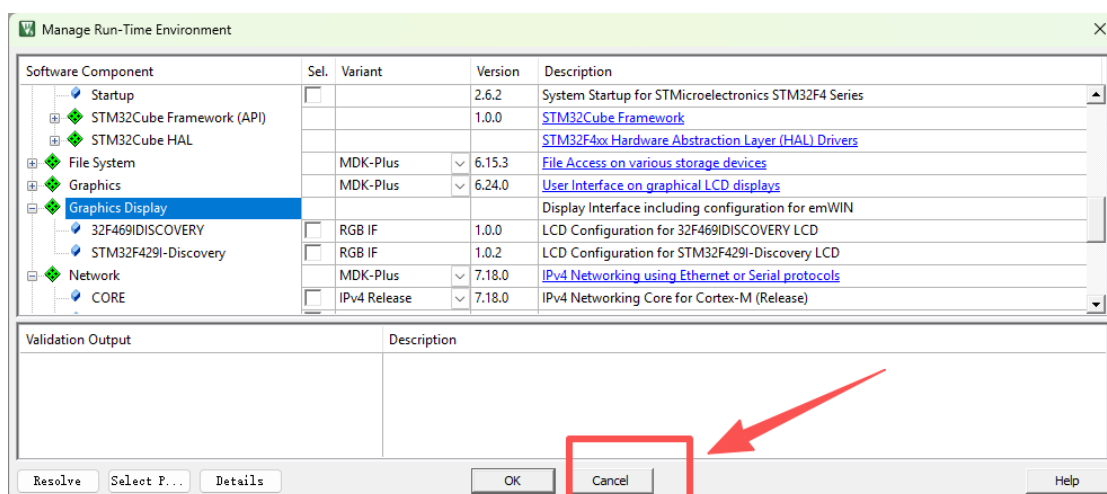


图 1.4 Manage Run-Time Environment 对话框

(4) 现在打开 USER ，目录下面包含 2 个文件夹和 2 个文件，如下图 1.5 所示：

名称	修改日期	类型	大小
DebugConfig	2026/6/28 下午 3:09	文件夹	
Listings	2026/6/28 下午 3:12	文件夹	
Objects	2026/6/28 下午 3:12	文件夹	
Template.uvoptx	2026/6/28 下午 3:09	UVOPTX 文件	
Template	2026/6/28 下午 3:09	ARMion5 Project	

图 1.5 USER 文件夹

Template.uvprojx 是工程文件，非常关键，不能轻易删除。Listings 和 Objects 文件夹是 MDK 自动生成的文件夹，用于存放编译过程产生的中间文件。这两个文件夹我们并不使用，所以删除。之后设置路径，存放于其他文件夹。

(5) 拷贝文件

将官方的固件库包里的源码文件复制到我们的工程目录文件夹下面。

1) 打开官方固件库包，定位到我们之前准备好的固件库包的目录

STM32F4xx_DSP_StdPeriph_Lib_V1.8.0\Libraries\STM32F4xx_StdPeriph_Driver 下面，将目录下面的 src,inc 文件夹复制到我们刚才建立的 FWLib 文件夹下面。src 存放的是固件库的.c 文件，inc 存放的是对应的.h 文件。

2) 将固件库包里面相关的启动文件复制到我们的工程目录 CORE 之下。打开官方固件库包，定位到目录

STM32F4xx_DSP_StdPeriph_Lib_V1.8.0\Libraries\CMSIS\Include 下面，将文件 core_cm4 , core_cmSimd , core_cmFunc 和文件 core_cmInstr 复制到 CORE 下面去。然后定位到目录

STM32F4xx_DSP_StdPeriph_Lib_V1.8.0\Libraries\CMSIS\Device\ST\STM32F4xx\Source\Templates\arm 下面，将里面的文件 startup_stm32f40_41xxx.s 复制到 CORE 下面。（不同容量的芯片使用不同的启动文件，可以根据芯片容量选择这个启动文件。）

3) 定位到目录：

STM32F4xx_DSP_StdPeriph_Lib_V1.8.0\Libraries\CMSIS\Device\ST\STM32F4xx\Include 下面将里面的 2 个文件 stm32f4xx , system_stm32f4xx 复制到我们的 USER 目录之

下。

4)把

STM32F4xx_DSP_StdPeriph_Lib_V1.8.0\Project\STM32F4xx_StdPeriph_Templates 下面的 6 个 文件 main.c, stm32f4xx_conf.h, stm32f4xx_it.c, stm32f4xx_it.h , main.h, system_stm32f4xx.c 复制到 USER 目 录下面。

5) 打开实验提供的 SYSTEM 文件夹，里面有三个子文件夹分别为 delay,sys,usart，每 个子文件夹下面都有相应的.c 文件和.h 文件。我们接下来要将这三个目录下面的代码加入 到我们工程中去。

名称	修改日期
inc	2026/6/28 下午 5:40
src	2026/6/28 下午 5:40

图 1.6 FWLib 目录文件浏览

名称	修改日期	类型	大小
core_cm4	2026/6/28 下午 3:43	C Header 源文件	112 KB
core_cmFunc	2026/6/28 下午 3:43	C Header 源文件	18 KB
core_cmInstr	2026/6/28 下午 3:43	C Header 源文件	28 KB
core_cmSimd	2026/6/28 下午 3:43	C Header 源文件	23 KB
startup_stm32f40_41xxx.s	2026/6/28 下午 3:43	S 文件	29 KB

图 1.7 Core 目录文件浏览

名称	修改日期	类型	大小
RTE	2026/6/28 下午 9:07	文件夹	
main	2026/6/28 下午 10:46	C 源文件	3 KB
main	2026/6/28 下午 3:43	C Header 源文件	2 KB
stm32f4xx	2026/6/28 下午 9:03	C Header 源文件	919 KB
stm32f4xx_conf	2026/6/28 下午 3:43	C Header 源文件	6 KB
stm32f4xx_it	2026/6/28 下午 9:50	C 源文件	4 KB
stm32f4xx_it	2026/6/28 下午 3:43	C Header 源文件	2 KB
system_stm32f4xx	2026/6/28 下午 3:43	C 源文件	60 KB
system_stm32f4xx	2026/6/28 下午 3:43	C Header 源文件	2 KB
template.uvguix.86177	2026/6/29 下午 8:31	86177 文件	93 KB
template.uvoptx	2026/6/28 下午 10:41	UVOPTX 文件	26 KB
template	2026/6/28 下午 10:41	碓ision5 Project	25 KB

图 1.8 User 目录文件浏览

名称	修改日期	类型
usart	2026/6/28 下午 6:04	文件夹
sys	2026/6/28 下午 6:04	文件夹
delay	2026/6/28 下午 6:04	文件夹

图 1.9 SYSTEM 目录文件浏览

(6) 添加文件加入工程:

打开 MDK，右键点击 Target1，选择 Manage Project Items。(也可以点击上框的 Manage Project Items。)

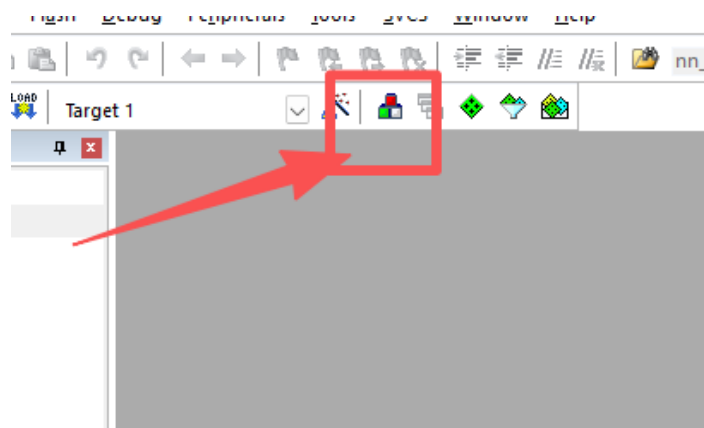


图 1.10 点击 Management Project Itmes

在 Project Targets 一栏，我们将 Target 名字修改为 Template，然后在 Groups 一栏删掉一个 Source Group1，建立 4 个 Groups: USER,CORE,FWLIB, SYSTEM。然后点击 OK，可以看到我们的 Target 名字以及 Groups 情况。

下面我们往 Group 里面添加我们需要的文件。选择需要添加文件的 Group，这里第一步我们选择 FWLIB，然后点击右边的 Add Files, 定位到我们刚才建立的目录 STM32F4xx_FWLib/src 下面，将里面所有的文件选中 (Ctrl+A)，然后点击 Add，然后 Close. 可以看到 Files 列表下面包含我们添加的文件。(这里需要说明一下，对于我们写代码，如果我们只用到了其中的某个外设，我们就可以不用添加没有用到的外设的库文件。例如我只用 GPIO，我可以只用添加 stm32f4xx_gpio.c 而其他的可以不用添加。这里我们全部添加进来是为了后面方便，不用每次添加，当然这样的坏处是工程太大，编译起来速度慢，我们可以自行选

择。) 用同样的方法, 将 Groups 定位到 CORE , USER 和 SYSTEM 下面, 添加需要的文件。CORE 下面需要添加的文件为 startup_stm32f40_41xxx.s, core_cm4 (注意, 默认添加的时候文件类型为.c,也就是添加 startup_stm32f40_41xxx.s 启动文件的时候, 你需要选择文件类型为 All files 才能看得到这个文件), USER 目录下面需要添加的文件为 main.c, stm32f4xx_it.c, system_stm32f4xx.c. SYSTEM 目录下面需要添加的文件为 sys.c, delay.c, usart.c。

这样我们需要添加的文件已经添加到我们的工程中了, 最后点击 OK, 回到工程主界面。

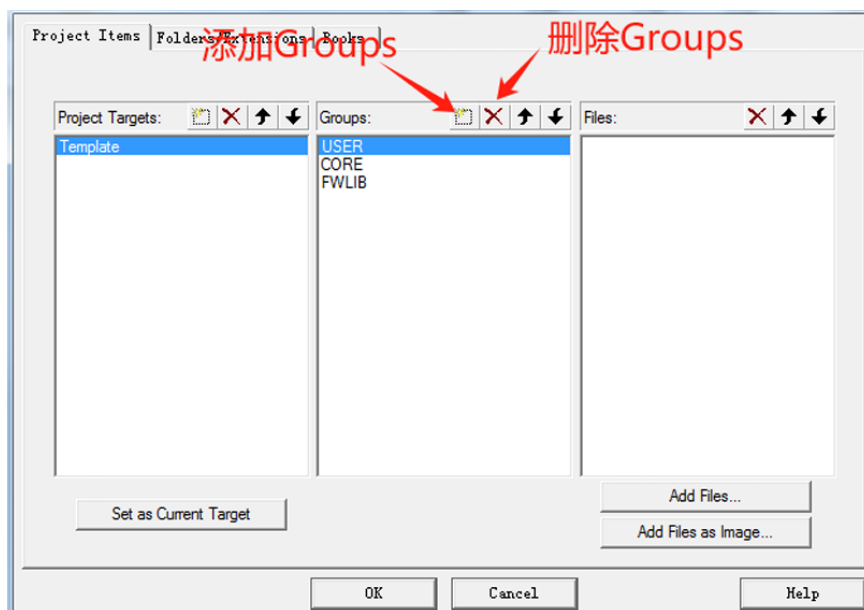


图 1.11 新建分组

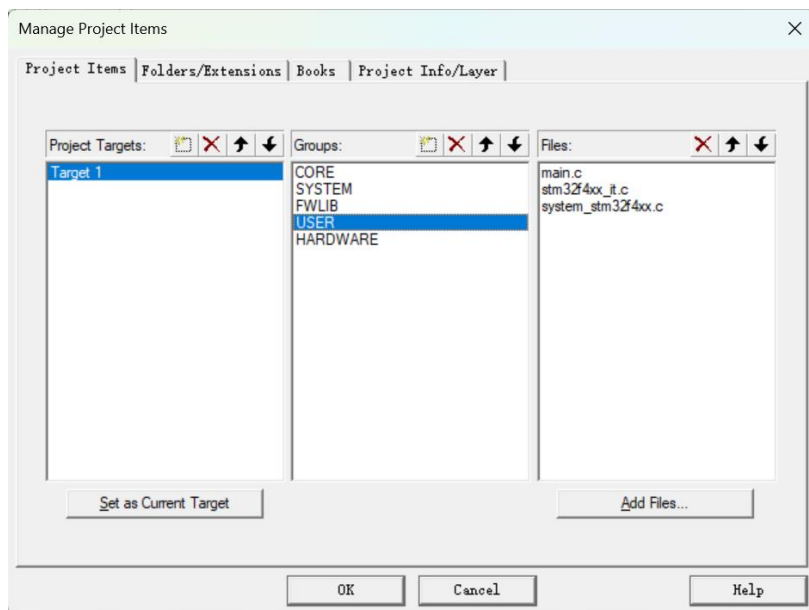


图 1.12 添加到 USER 分组

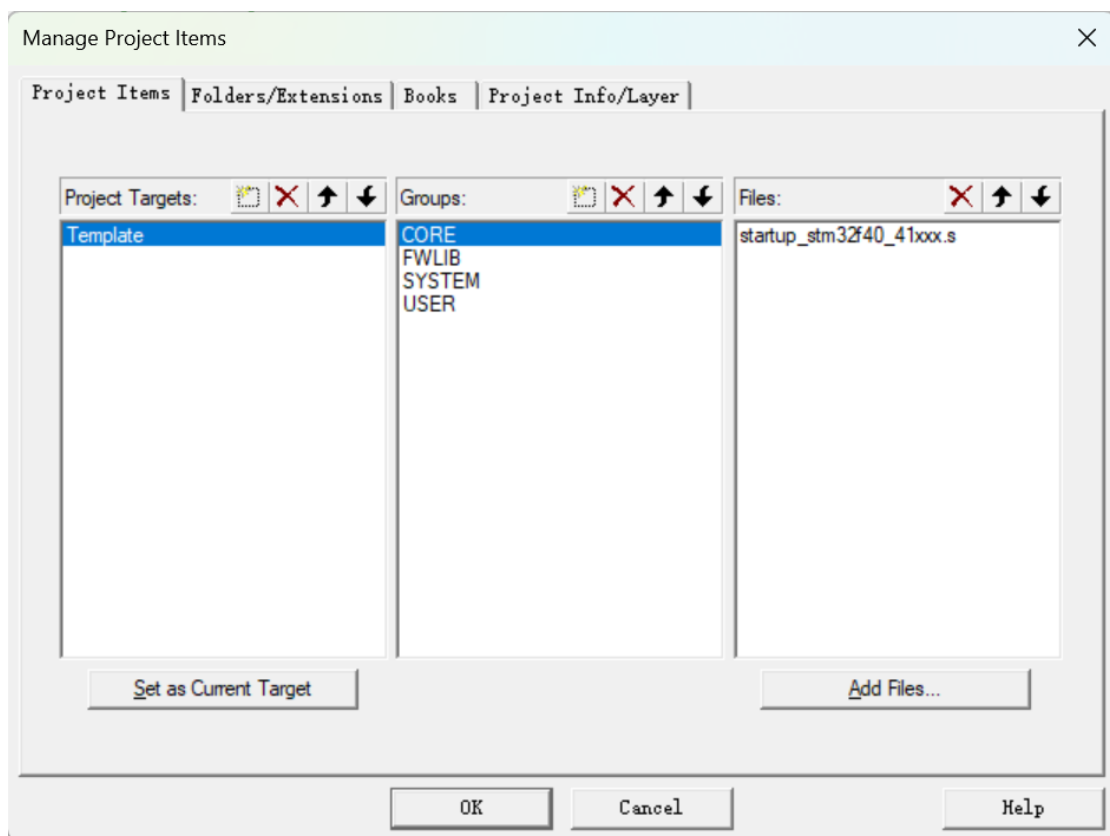


图 1.13 添加到 CORE 分组

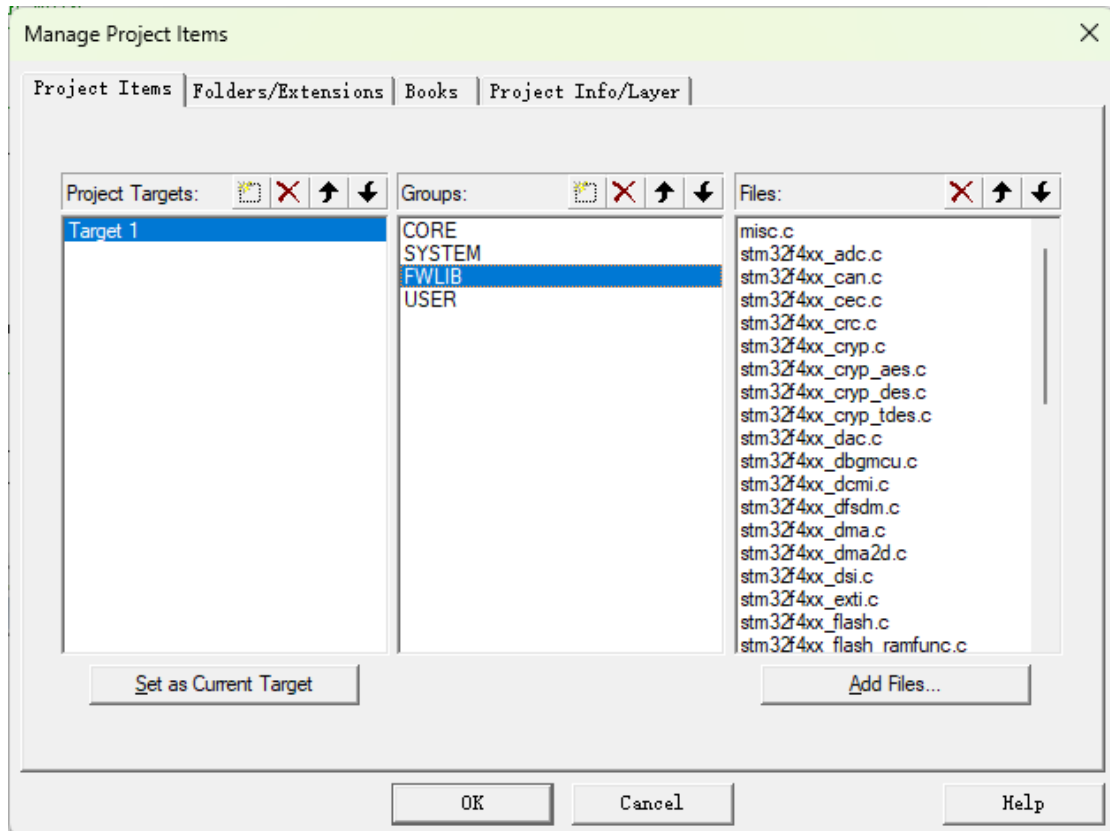
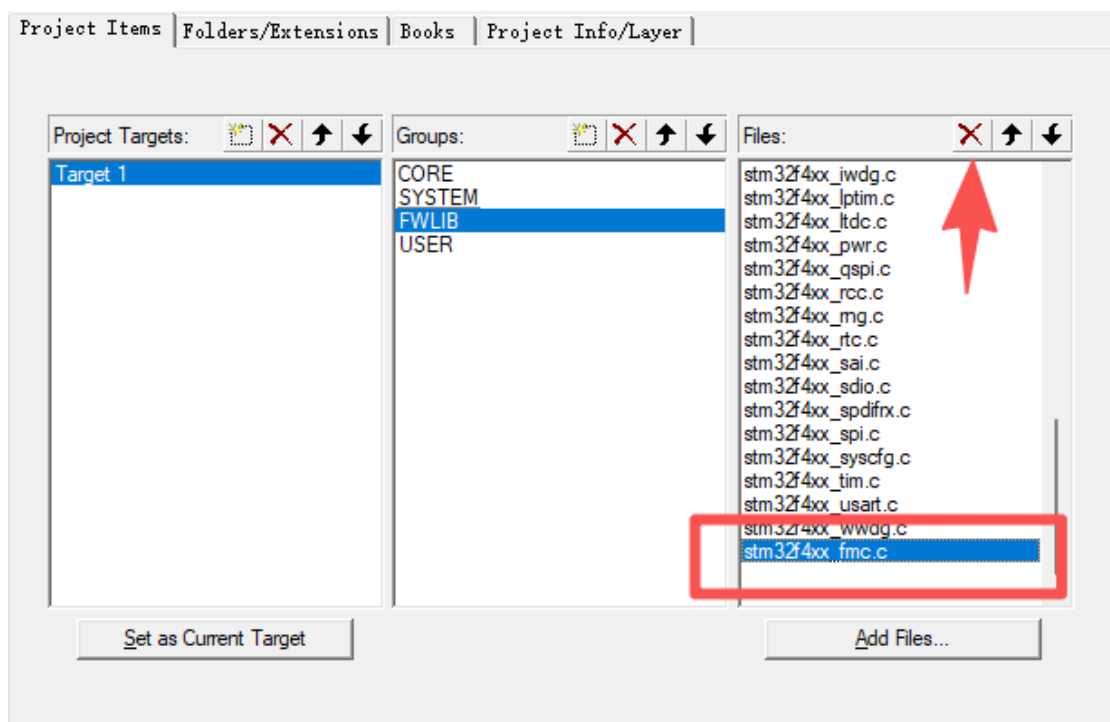


图 1.14 添加到 FWLIB 分组

这里需要注意，FWLIB 之前是添加的所有文件，这里需要手动删掉，如下图所示：



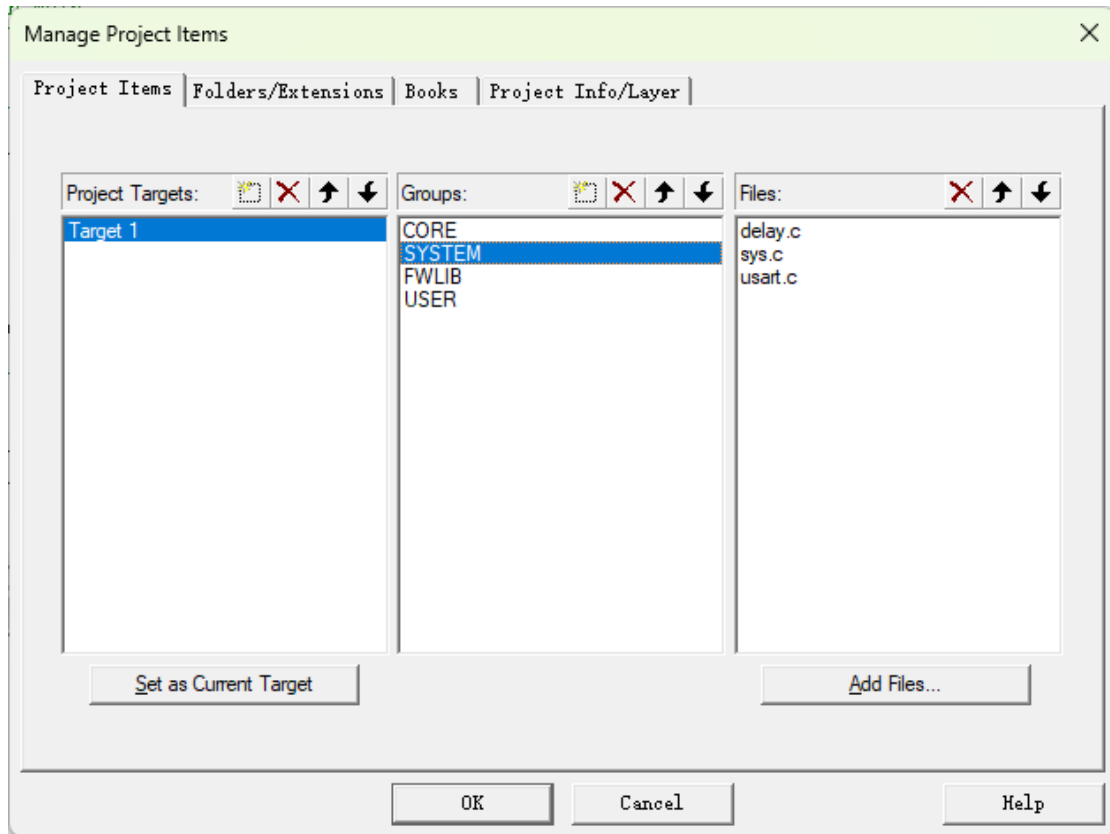


图 1.15 添加到 SYSTEM 分组

(7) 设置 Output 路径 选择编译中间文件编译后存放目录。点击魔术棒，然后选择“Output”选项下面的“Select folder for objects...”,然后选择目录为上面新建的 OBJ 目录。这里大家注意，如果我们不设置 Output 路径，那么默认的编译中间文件存放目录就是 MDK 自动生成的 Objects 目录 和 Listings 目录。

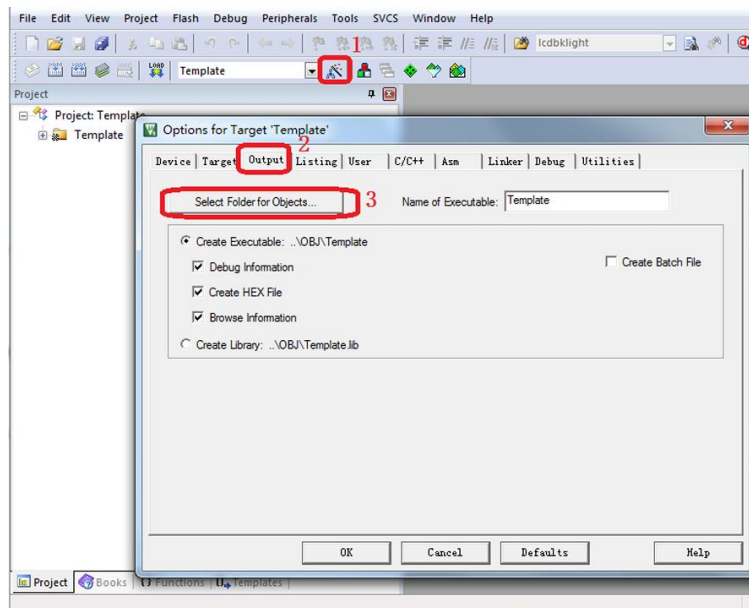


图 1.16 选择编译后的文件存放目录

(8) 设置头文件路径 对于任何一个工程，都需要把工程中引用到的所有头文件的路径都包含到进来。回到工程主菜单，点击魔术棒，点击 c/c++选项.然后点击 Include Paths 右边的按钮。弹出一个 添加 path 的对话框，然后将图 1.17 上面的目录添加进去。keil 只会在一级目录查找，所以如果你的目录下面还有子目录，记得 path 一定要定位到最后一级子目录，然后点击 OK。

(9) 配置全局宏定义变量 配置一个全局的宏定义变量。定位到 c/c++界面，然后填写“STM32F40_41xxx,USE_STDPERIPH_DRIVER”到 Define 输入框里面，language c 设置为 c99，如图 1.19 所示。

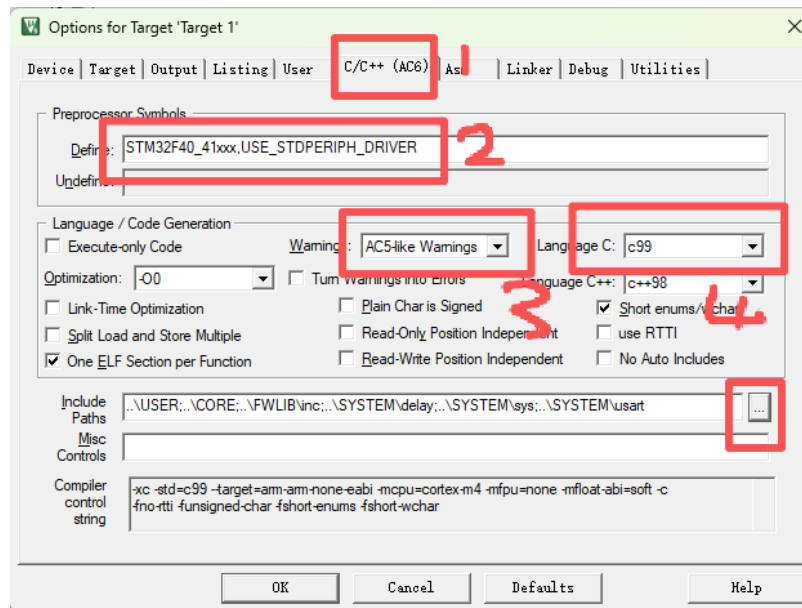


图 1.17 C/C++选项卡

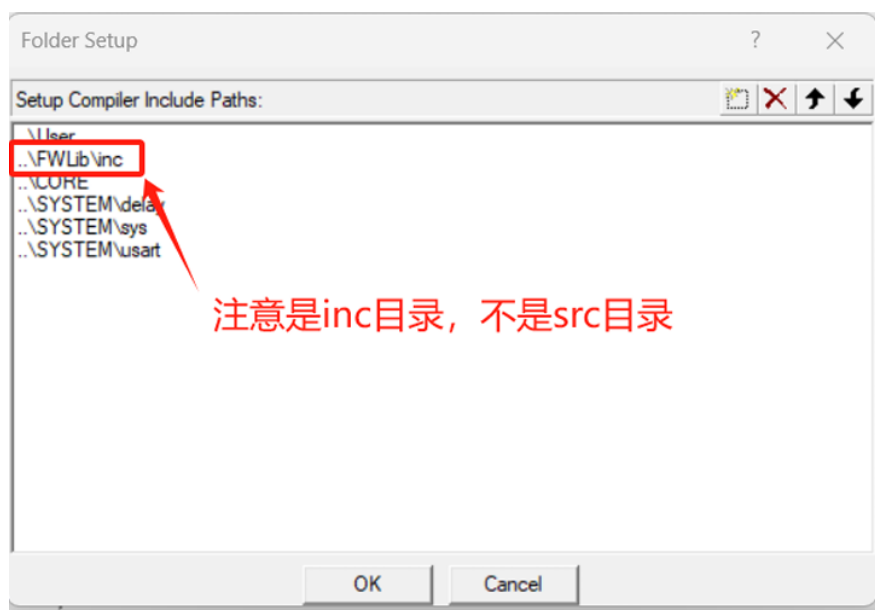


图 1.18 添加头文件路径到 PATH

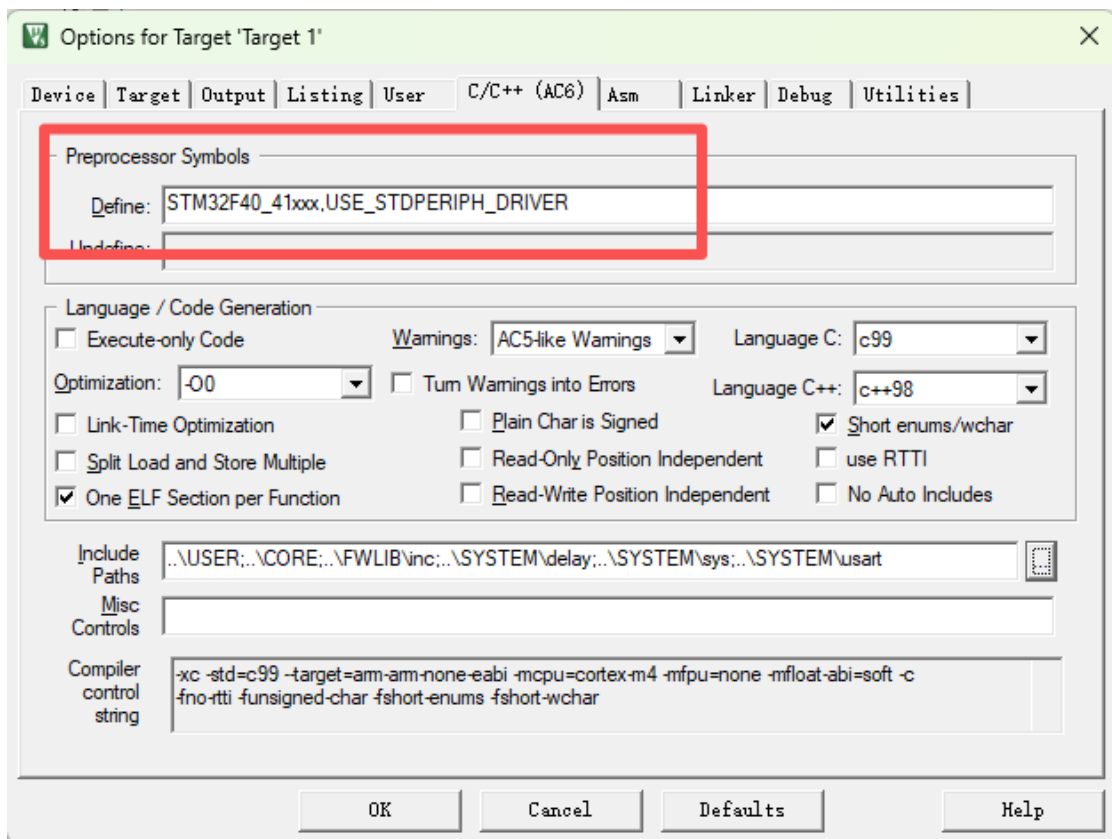


图 1.19 添加全局宏定义标识符到 Define 输入框

这样一个工程模版建立完毕。下面还需要配置，让编译之后能够生成 hex 文件。

(10) 点击魔术棒，进入配置菜单，选择 Output。然后勾上下面三个选项。其中 Create HEX file 是编译生成 hex 文件，Browser Information 是可以查看变量和函数定义。

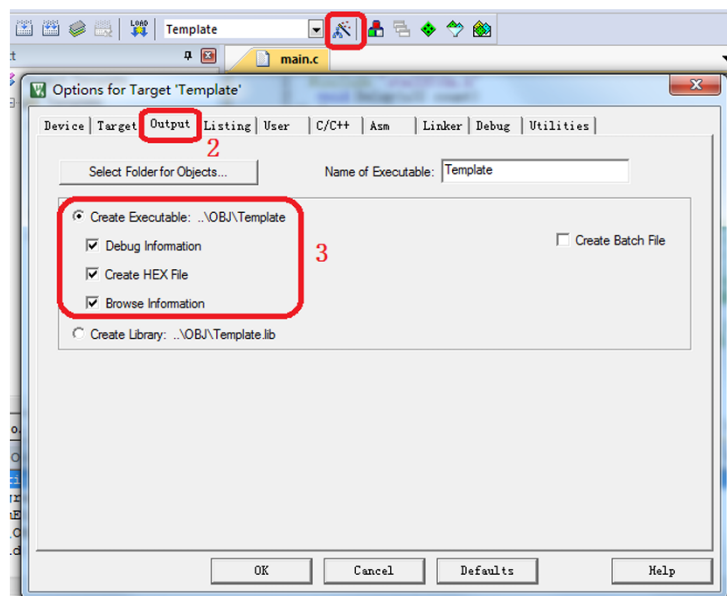


图 1.20 Output 选项卡设置

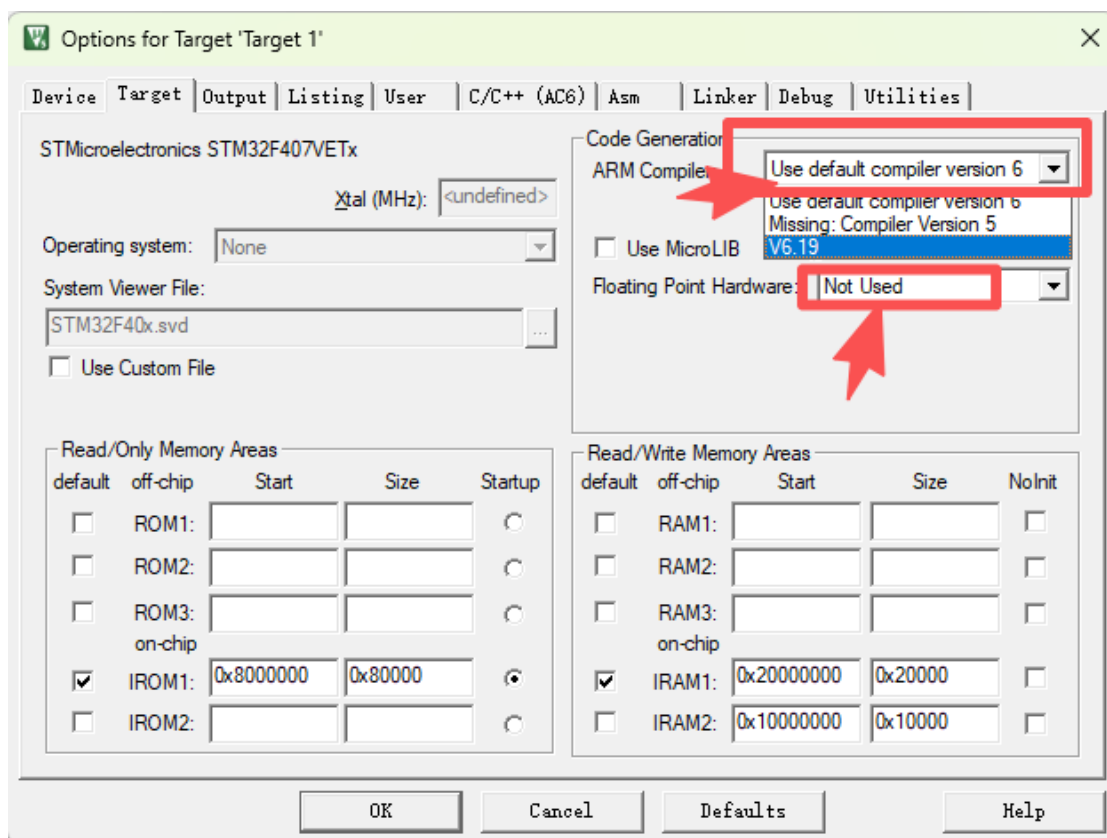


图 1.21 TARGET 设置

再切换到 TARGET 页面，选择编译器版本，这里可以选择 Use default compiler version 6 或者 V6.19 均可，如果这两个版本都没有，则参考链接

<https://www.cnblogs.com/chicheng/p/17524580.html> 下载，下面的 float point hardware 修改为 not used。

注意事项，由于 F407 的固件与 KEIL 版本存在冲突，下面还有几个地方需要修改：

注释掉下面的代码：

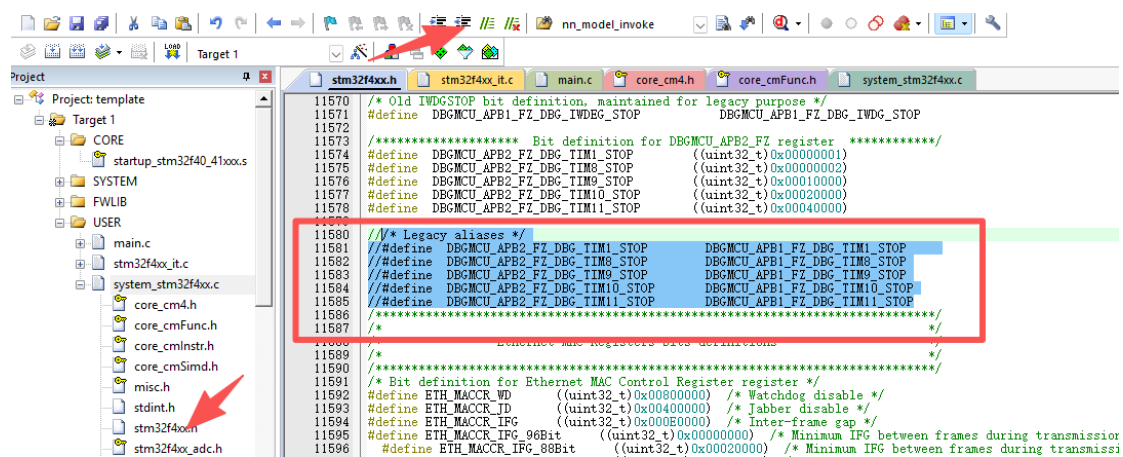


图 1.21 修改报错文件

修改 `stm32f4xx_it.c` 文件，找到 134 行的这个函数，将函数内部的 `TimingDelay_Decrement()`;这句代码删除，最后点击保存。

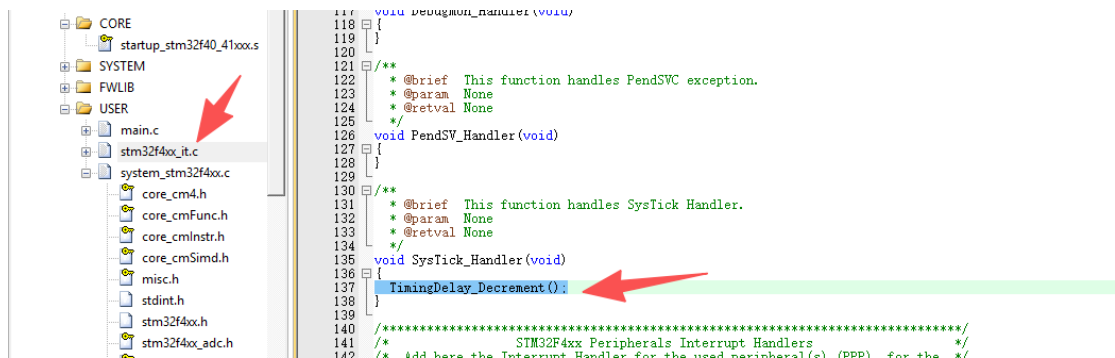


图 1.22 删除报错代码

点击左上角的 build 编译代码编译代 码，可以看到编译成功，没有报错，并且生成了 hex 文件在 OBJ 目录下。

```
Build Output
compiling stm32f10x_rtc.c...
compiling stm32f10x_sdio.c...
compiling stm32f10x_spi.c...
compiling stm32f10x_tim.c...
compiling stm32f10x_usart.c...
compiling delay.c...
compiling stm32f10x_wwdg.c...
compiling usart.c...
compiling sys.c...
linking...
Program Size: Code=1372 RO-data=268 RW-data=8 ZI-data=1832
FromELF: creating hex file...
"..\\Template.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:04
```

图 1.23 编译结果图

(11) 下载程序 在下载程序之前需要将 stlink 与小板子连接起来，如图 1.25。还需要下载 stlink 的驱动， 驱动安装如图 1.24 所示，进入驱动文件夹，双击 dpinst_amd64.exe 即可安装。

Keil5 MDK5.39(64bit)	2024/6/1 20:22	文件夹
ST LINK驱动及教程	2023/8/2 15:38	文件夹

amd64	2014/1/21 17:16	文件夹	
x86	2014/1/21 17:16	文件夹	
dpinst_amd64.exe	2010/2/9 4:36	应用程序	665 KB
dpinst_x86.exe	2010/2/9 3:59	应用程序	540 KB
stlink_dbg_winusb.inf	2014/1/21 17:03	安装信息	4 KB
stlink_VCP.inf	2013/12/10 21:08	安装信息	3 KB
stlink_winusb_install.bat	2013/5/15 22:33	Windows 批处理...	1 KB
stlinkdbgwinusb_x64.cat	2014/1/21 17:14	安全目录	11 KB
stlinkdbgwinusb_x86.cat	2014/1/21 17:14	安全目录	11 KB
stlinkvcp_x64.cat	2013/12/10 21:08	安全目录	9 KB
stlinkvcp_x86.cat	2013/12/10 21:09	安全目录	9 KB

图 1.24 驱动安装

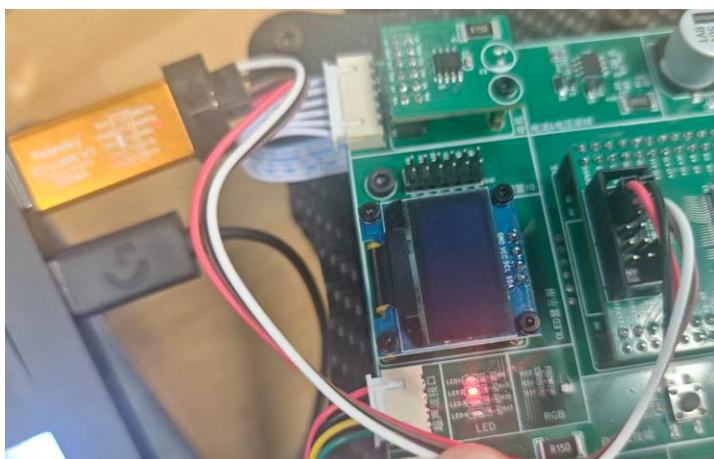


图 1.25 stlink 与小板子的连线图

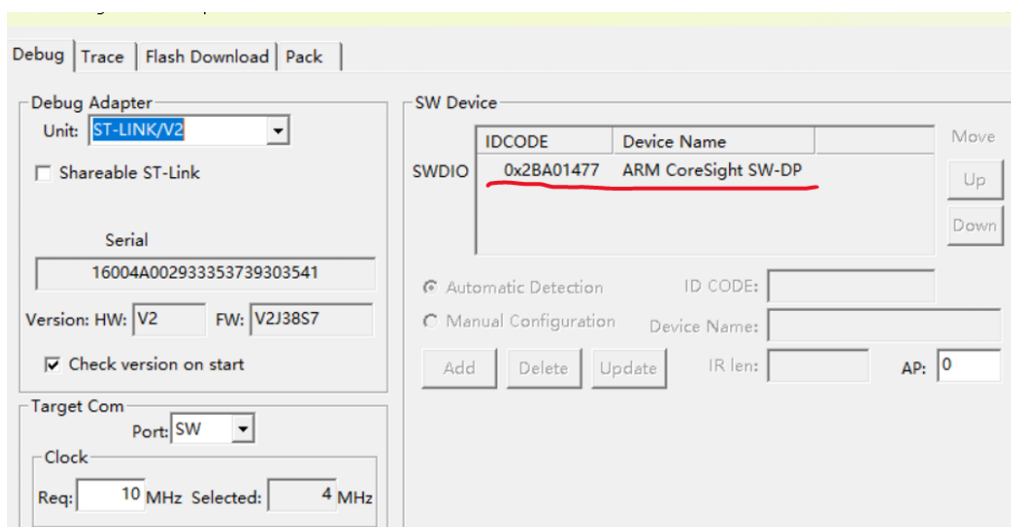
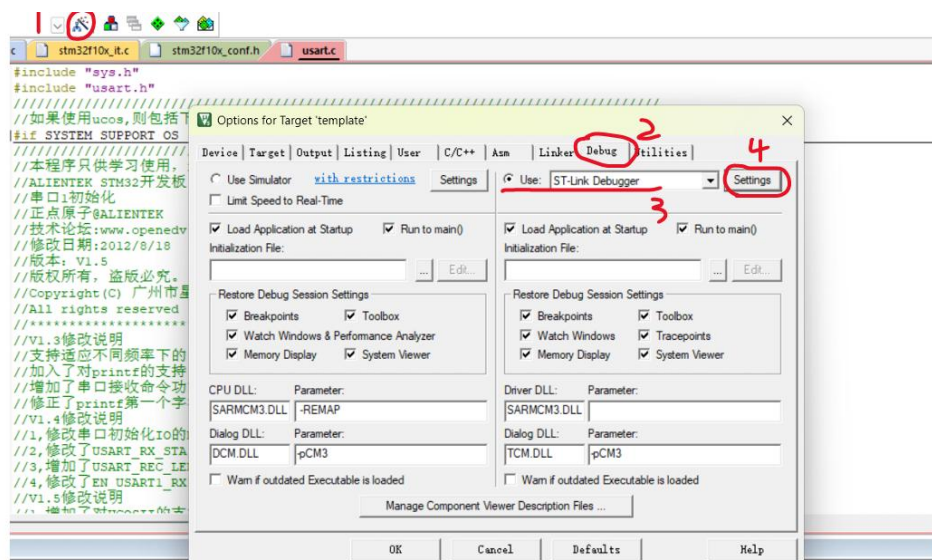


图 1.26 stlink 配置

确定识别到 ST-link 下载器，先编译完程序，再点击下载图标，即完成程序的下载。



图 1.27 编译下载程序

实验二 LED 灯流水灯实验

2.1 实验目标

STM32 最简单的外设是 IO 口的高低电平控制，本小节将首先通过一个最基础的跑马灯 程序，带大家开启 STM32F407 之旅，通过本次的学习，希望同学们了解并掌握 STM32F407 的 IO 口作为输出使用的方法。

2.2 实验简介

2.2.1 STM32 IO 简介

(1) 在 STM32F407VET6 的标准外设库开发中，GPIO 端口操作对应的库函数以及相关定义主要位于 `stm32f4xx_gpio.h` 和 `stm32f4xx_gpio.c` 文件中。通过调用标准外设库提供的 GPIO 初始化函数和端口读写函数，可以完成 IO 口输入、输出、复用功能以及上下拉等配置。

(2) STM32F407VET6 属于 STM32F4 系列微控制器，其 GPIO 控制方式与 STM32F1 系列有所不同。STM32F1 中常见的 CRL、CRH 配置寄存器在 STM32F4 中不再使用，STM32F4 的每个 GPIO 端口主要通过以下寄存器进行控制：端口模式寄存器 MODER、输出类型寄存器 OTYPER、输出速度寄存器 OSPEEDR、上拉/下拉寄存器 PUPDR、输入数据寄存器 IDR、输出数据寄存器 ODR、位设置/复位寄存器 BSRR、配置锁定寄存器 LCKR，以及复用功能寄存器 AFRL 和 AFRH。其中，MODER 用于配置引脚为输入、输出、复用或模拟模式；OTYPER 用于设置推挽输出或开漏输出；OSPEEDR 用于设置 IO 输出速度；PUPDR 用于配置上拉、下拉或无上下拉状态；IDR 和 ODR 分别用于读取输入状态和设置输出状态。

(3) 实验固件库相关函数如下。

① 在标准外设库开发中，配置 STM32F407VET6 的 GPIO 模式、输出类型、输出速度以及上下拉状态，通常通过 GPIO 初始化函数完成：

```
void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct);
```

第一个参数 GPIOx 用于指定 GPIO 端口，例如 GPIOA、GPIOB、GPIOC、

GPIO_D、GPIO_E 等。需要注意的是，标准外设库中支持的 GPIO 端口范围较广，但 STM32F407VET6 实际可使用的端口和引脚应以芯片封装和开发板原理图为准。第二个参数为 GPIO 初始化结构体指针，结构体类型为 GPIO_InitTypeDef。该结构体中通常包含 GPIO_Pin、GPIO_Mode、GPIO_Speed、GPIO_OType 和 GPIO_PuPd 等成员，分别用于设置引脚编号、工作模式、输出速度、输出类型和上下拉方式。

② 在固件库中，读取 GPIO 输入端口数据通常通过 GPIO_ReadInputDataBit 函数实现。该函数用于读取指定 GPIO 引脚的输入电平状态，返回值为 Bit_SET 或 Bit_RESET：

```
uint8_t GPIO_ReadInputDataBit(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
```

例如，当按键连接到某个 GPIO 输入引脚时，可以通过该函数判断按键是否被按下。

③ 在固件库中，设置 GPIO 输出端口的整体输出状态可以通过 GPIO_Write 函数实现。该函数一般用于一次性向某个 GPIO 端口的多个引脚写入数据：

```
void GPIO_Write(GPIO_TypeDef* GPIOx, uint16_t PortVal);
```

该函数会将 PortVal 的值写入对应 GPIO 端口的输出数据寄存器 ODR，从而控制多个引脚的输出状态。

④ 在实际实验中，更常用的是通过 GPIO_SetBits() 和 GPIO_ResetBits() 函数控制某一个或多个 GPIO 引脚的高低电平输出：

```
void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
```

```
void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin);
```

其中，GPIO_SetBits() 用于将指定引脚置为高电平，GPIO_ResetBits() 用于将指定引脚置为低电平。这两个函数本质上是通过操作 GPIO 的位设置/复位寄存器 BSRR 来实现引脚电平控制。由于该方式操作简单、执行效率较高，并且不会影响同一端口的其他引脚，因此在 LED 控制、蜂鸣器控制、继电器控制等实验中应用较多。

2.2.1 硬件设计

四路 led 灯电路原理图如图 2.1 所示，该电路为 STM32F407VET6 开发板上的 LED 指示灯电路，共包含 LED1、LED2、LED3、LED4 四个发光二极管。左侧为 3.3V 电源，每一路 LED 都通过一个 $1.3\text{k}\Omega$ 的限流电阻与 3.3V 相连，右侧的 LED1、LED2、LED3、LED4 分别连接到单片机的 GPIO 引脚。

电路工作时，电流从 3.3V 出发，经过限流电阻，再经过 LED，最后流向单片机 IO 引脚。由于 LED 的另一端接到单片机引脚，因此该电路通常为低电平点亮方式：当对应 GPIO 引脚输出低电平时，电流形成通路，LED 点亮；当 GPIO 引脚输出高电平时，LED 两端电压差不足，LED 熄灭。

其中 R9、R13、R15、R17 的作用是限制通过 LED 的电流，防止电流过大烧坏 LED 或单片机 IO 口。四个 LED 电路结构相同，可以分别由单片机的四个 GPIO 引脚独立控制，用于显示程序运行状态、按键状态或其他实验现象。

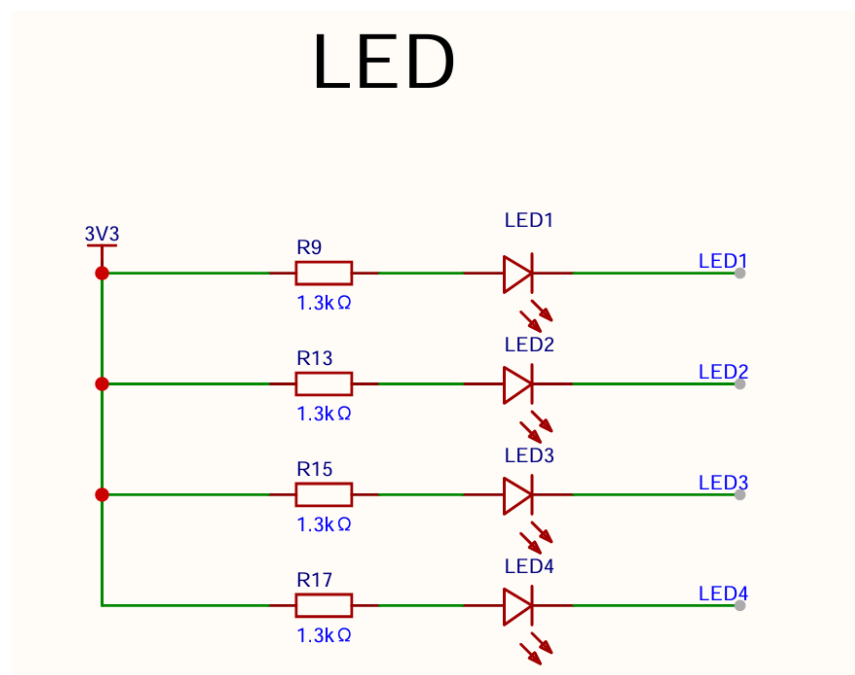


图 2.1 四路 led 灯电路原理图

2.3 实验过程

STM32F407VET6 的 IO 操作步骤较为简单，主要包括三个部分：首先使能对应 GPIO 端口的时钟；然后初始化 GPIO 引脚的工作模式；最后通过 GPIO 输出高低电

平控制 LED 的亮灭。与 STM32F103 不同，STM32F407VET6 的 GPIO 挂载在 AHB1 总线上，因此使能 GPIO 时钟时应调用 `RCC_AHB1PeriphClockCmd()` 函数，而不是 `RCC_APB2PeriphClockCmd()` 函数。

根据 LED 原理图可知，LED1、LED2、LED3、LED4 的一端通过 1.3kΩ 限流电阻连接到 3.3V 电源，另一端连接到单片机 GPIO 引脚。因此该 LED 电路属于低电平点亮方式。当单片机对应 GPIO 引脚输出低电平时，电流由 3.3V 经过限流电阻和 LED 流入 GPIO 引脚，LED 点亮；当 GPIO 引脚输出高电平时，LED 两端电压差不足，LED 熄灭。

具体操作步骤如下。

(1) 复制前面实验所建立的 STM32F407VET6 工程模板文件夹，并将其重命名为“LED 灯闪烁”。

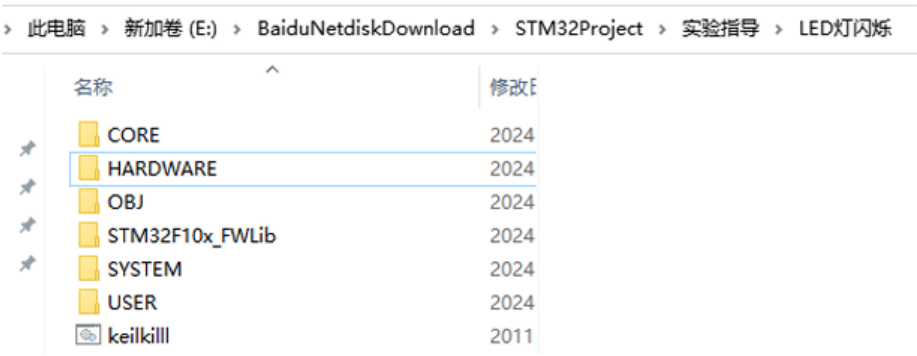


图 2.2 新建文件夹

(2) 打开“LED 灯闪烁”工程文件夹，新建一个文件夹，命名为 `HARDWARE`。双击进入 `HARDWARE` 文件夹后，再新建一个名为 `LED` 的文件夹，用于存放 LED 相关的驱动文件。

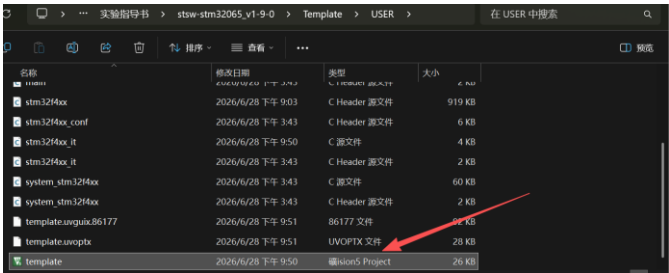


图 2.2 打开工程文件

(3) 在 `HARDWARE/LED` 文件夹中新建两个文件，分别命名为 `led.c` 和 `led.h`。其中，`led.h` 用于声明 LED 初始化函数和 LED 控制宏定义，`led.c` 用于编写 LED 初始化函数。



图 2.3 创建 `led.c` 和 `led.h` 文件

(4) 将 `led.c` 添加到 Keil 工程中。可在工程左侧文件管理区域中新建一个分组，命名为 `HARDWARE` 或 `LED`，然后右键选择 `Add Existing Files to Group`，将 `led.c` 添加进去，如图 2.3 所示。

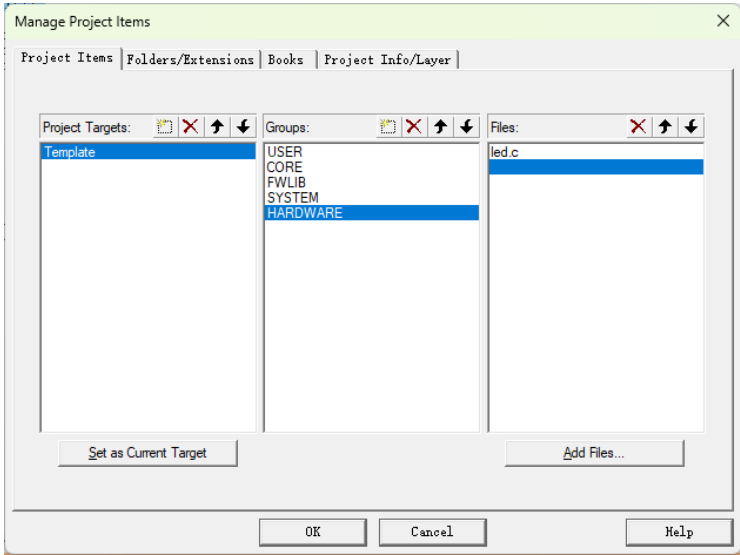


图 2.4 新建一个 `led` 分组

(5) 在 Keil 中添加头文件路径。点击魔术棒图标，进入 `Options for Target`，选择 `C/C++` 选项卡，在 `Include Paths` 中添加 `LED` 文件夹路径，例如：

`..\HARDWARE\LED`，如图 2.5 所示

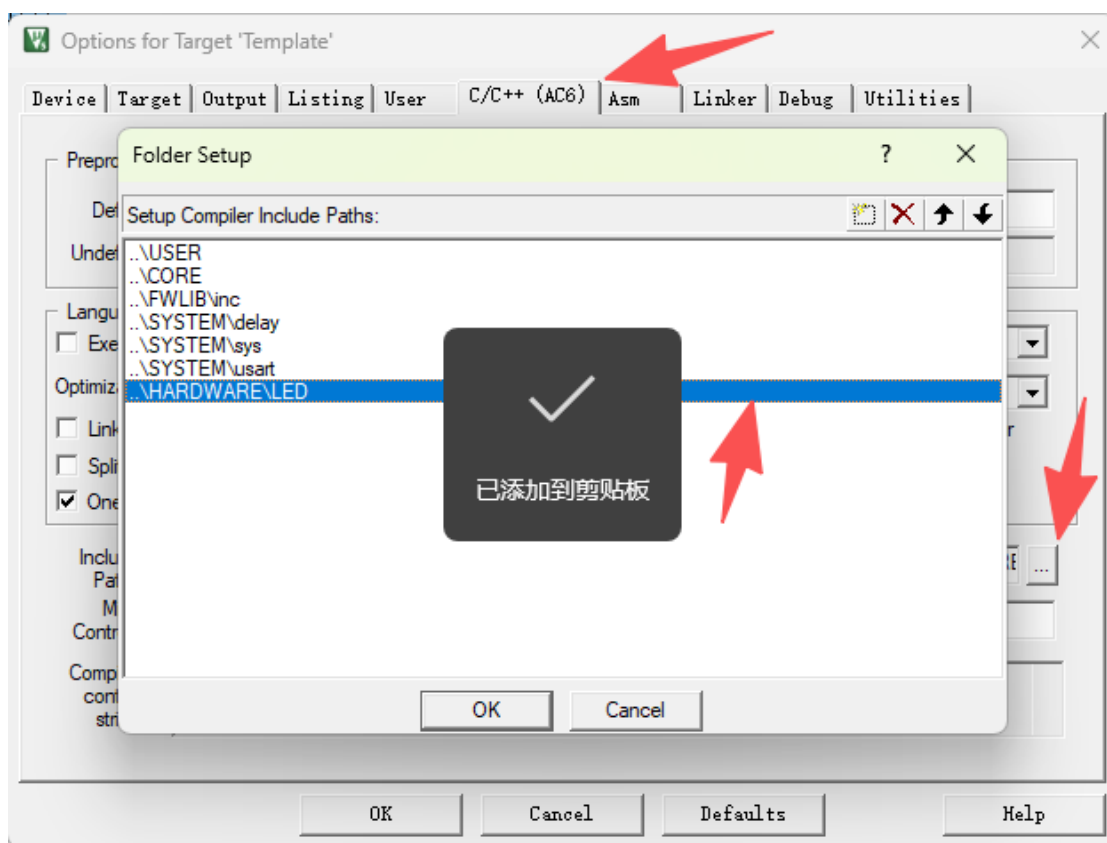


图 2.5 添加 LED.H 路径

如果工程中还使用了 SYSTEM/delay、USER、CORE、FWLib/inc 等文件夹，也需要同时添加对应路径。

(6) 编写 LED 初始化函数。首先调用 RCC_AHB1PeriphClockCmd() 使能 LED 所在 GPIO 端口的时钟；然后定义 GPIO_InitTypeDef 结构体变量，配置 GPIO 为普通推挽输出模式；最后调用 GPIO_Init() 完成 GPIO 初始化，如图 2.6 所示。


```

38 ~ /*
39 void LED_Init(void)
40 {
41     GPIO_InitTypeDef GPIO_InitStructure;
42
43     /*
44     * 第一步：打开 GPIOB 和 GPIOD 的时钟。
45     *
46     * LED1、LED3、LED4 接在 GPIOD；
47     * LED2 接在 GPIOB。
48     */
49     RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOB | RCC_AHB1Periph_GPIOD, ENABLE);
50
51     /*
52     * 第二步：配置 GPIO 输出模式。
53     *
54     * GPIO_Mode_OUT      : 普通输出模式，用来输出高低电平。
55     * GPIO_OType_PP       : 推挽输出，适合控制 LED。
56     * GPIO_Speed_50MHz    : GPIO 输出速度，控制 LED 足够使用。
57     * GPIO_PuPd_UP        : 上拉，默认更偏向高电平，也就是默认灭灯。
58     */
59     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
60     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
61     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
62     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
63
64     /* 第三步：初始化 LED2 的 PB3 引脚。 */
65     GPIO_InitStructure.GPIO_Pin = LED2_PIN;
66     GPIO_Init(LED2_PORT, &GPIO_InitStructure);
67
68     /* 第四步：初始化 LED1、LED3、LED4 的 PD7、PD5、PD6 引脚。 */
69     GPIO_InitStructure.GPIO_Pin = LED1_PIN | LED3_PIN | LED4_PIN;
70     GPIO_Init(LED1_PORT, &GPIO_InitStructure);
71
72     /* 第五步：初始化完成后先关闭全部 LED，避免上电后灯状态不确定。 */
73     LED_AllOff();
74 }
75

```

图 2.6 LED 引脚初始化

由于该电路为低电平点亮方式，所以初始化完成后，可先将 LED 对应的 GPIO 引脚置高，使 LED 默认处于熄灭状态。

(7) 在 main.c 中包含 led.h 头文件，并在主函数中调用 LED_Init() 初始化 LED。随后通过 GPIO_ResetBits() 或自定义宏控制对应 LED 点亮，通过 GPIO_SetBits() 控制对应 LED 熄灭。若要实现 LED 闪烁效果，可结合延时函数循环控制 LED 的亮灭状态。

```

103 | *   5. 关闭全部 LED，一轮流水结束。
104 | */
105 | void LED_FlowingWater(uint16_t interval_ms)
106 | {
107 |     LED_Alloff();
108 |     LED_Write(LED1_PORT, LED1_PIN, LED_ON);
109 |     delay_ms(interval_ms);
110 |
111 |     LED_Alloff();
112 |     LED_Write(LED2_PORT, LED2_PIN, LED_ON);
113 |     delay_ms(interval_ms);
114 |
115 |     LED_Alloff();
116 |     LED_Write(LED3_PORT, LED3_PIN, LED_ON);
117 |     delay_ms(interval_ms);
118 |
119 |     LED_Alloff();
120 |     LED_Write(LED4_PORT, LED4_PIN, LED_ON);
121 |     delay_ms(interval_ms);
122 |
123 |     LED_Alloff();
124 | }

```

图 2.7 流水灯核心代码

```

21 | int main(void)
22 | {
23 |     /*
24 |     * 初始化延时函数。
25 |     *
26 |     * delay_ms() 内部依赖 SysTick 计数。
27 |     * STM32F407 常用系统主频是 168MHz，所以这里填 168。
28 |     * 如果不先调用 delay_init(168)，LED_FlowingWater() 里的 delay_ms()
29 |     * 可能不能正常工作，现象就会像程序卡在第一个灯那里。
30 |     */
31 |     delay_init(168);
32 |
33 |     /*
34 |     * 初始化 4 个普通 LED 的 GPIO 引脚。
35 |     *
36 |     * LED_Init() 会完成：
37 |     * 1. 打开 GPIOB 和 GPIOD 的时钟；
38 |     * 2. 把 LED 引脚配置为普通推挽输出；
39 |     * 3. 先关闭全部 LED。
40 |     */
41 |     LED_Init();
42 |
43 |     /*
44 |     * 主循环。
45 |     * 单片机程序一般不会主动结束，所以这里用 while(1) 一直循环。
46 |     * LED_FlowingWater(300) 表示执行一轮流水灯：
47 |     * LED1 亮 300ms；
48 |     * LED2 亮 300ms；
49 |     * LED3 亮 300ms；
50 |     * LED4 亮 300ms。
51 |     */
52 |
53 |     while (1)
54 |     {
55 |         LED_FlowingWater(300);
56 |     }
57 | }

```

图 2.8 主函数

(8) 编译工程并下载到 STM32F407VET6 开发板中。下载成功后, 观察 LED1、LED2、LED3、LED4 是否按照程序设定依次点亮、熄灭或闪烁。如果 LED 没有点亮, 应重点检查 LED 对应的 GPIO 端口和引脚是否与原理图一致, 以及 GPIO 时钟是否正确使能。